Studio Nx

JS 114 — AudioEngine

2026-10-14

JS 114 — `AudioEngine` · `createClickPlayer()` (AudioContext 연결)

항목	내용
파일	`studionx/frontend/static/js/audio/engine.js` (새 파일)
작업	S1-05 오디오 엔진 기초: 열한 번째 함수 (FR-201~205 AudioContext 연결)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	데모 페이지(재생선·메트로놈 소리) 브라우저 확인, 그 뒤 S1-05 종료

1. 하는 일

104~113의 순수 계산과 `Transport`·`Metronome`을 **실제 `AudioContext`에 연결**합니다. `clock`은 `AudioContext.currentTime`, 타이머는 `setInterval`, 클릭 소리는 오실레이터로 냅니다.

대상	하는 일
`createClickPlayer(context, destination)`	짧은 사각파로 "딱" 소리를 내는 함수를 만듦. 강박은 높고 크게 (1600Hz)
`new AudioEngine({context, onStep, baseBpm, timeSigNum, ...})`	컨텍스트·트랜스포트·메트로놈을 묶음. `context`를 안 넘기면 `window.AudioContext` 사용
`start({fromTick, countIn})`	(브라우저) 컨텍스트를 깨우고 재생. `countIn`이면 한 마디 카운트인 뒤 0에서 시작
`stop() · pause() · resume() · seek(tick)`	재생 제어
`setTempo · setLoop · setMetronome · setTimeSignature`	트랜스포트·메트로놈에 전달
`positionTick()`	화면 재생선용 위치(tick)

페이지 사용 예(개념):

```
const engine = new AudioEngine({ metronomeEnabled: true, onStep: function (tick, time) { /* 악기 소리 */
} });
engine.start({ countIn: true });
requestAnimationFrame(function draw() { var t = engine.positionTick(); /* 재생선 그리기 */ requestAnimati
onFrame(draw); });
```

2. 코드

```
export function createClickPlayer(context, destination) {
  if (destination == null) destination = context.destination;
  return function playClick(time, accent) {
    const osc = context.createOscillator();
    const gain = context.createGain();
    osc.type = 'square';
    osc.frequency.value = accent ? 1600 : 1000;
    const dur = 0.03;
    gain.gain.setValueAtTime(accent ? 0.3 : 0.15, time);
```

```
    gain.gain.exponentialRampToValueAtTime(0.0001, time + dur);
    osc.connect(gain);
    gain.connect(destination);
    osc.start(time);
    osc.stop(time + dur + 0.01);
  };
}

export class AudioEngine {
  constructor(options) {
    options = options || {};
    let context = options.context != null ? options.context : null;
    if (context == null) {
      const Ctor = (typeof window !== 'undefined') ? (window.AudioContext || window.webkitAudioContext)
: null;
      if (Ctor == null) throw new Error('AudioContext 를 만들 수 없습니다 — options.context 를 넘겨 주세
요');
      context = new Ctor();
    }
    this.context = context;
    this.destination = options.destination != null ? options.destination : context.destination;
    const self = this;
    this.clock = { now: function () { return self.context.currentTime; } };
    this.onStep = typeof options.onStep === 'function' ? options.onStep : function () {};
    this.metronome = new Metronome({ click: createClickPlayer(context, this.destination), enabled: …, ti
meSigNum: …, timeSigDen: … });
    this.transport = new Transport({ clock: this.clock, …, onStep: function (tick, time) { self._onStep
(tick, time); } });
  }
  _onStep(tick, time) { this.metronome.step(tick, time); this.onStep(tick, time); }
  start(options) {
    options = options || {};
    this._resumeContext();
    let from = options.fromTick != null ? options.fromTick : 0;
    if (options.countIn) from = -countInTicks(this.metronome.timeSigNum, this.metronome.timeSigDen);
    this.transport.start(from);
    return this;
  }
  _resumeContext() {
    const ctx = this.context;
    if (ctx != null && ctx.state === 'suspended' && typeof ctx.resume === 'function') { try { ctx.resume
(); } catch (e) {} }
  }
}
```

최신 문법으로 쓰면 (공부용)

```
const Ctor = globalThis.AudioContext ?? globalThis.webkitAudioContext;
const from = options.fromTick ?? 0;
ctx.resume?.().catch(() => {});
```

최신 문법	실제 코드	뜻
`??` (널 병합)	`options.fromTick != null ? options.fromTick : 0`	null/undefined일 때만 기본값
선택적 호출 `resume?.()`	`typeof ctx.resume === 'function'` 확인	함수가 있을 때만 호출

최신 문법	실제 코드	뜻
`globalThis`	`typeof window !== 'undefined'`	브라우저 전역을 안전하게 봄

3. 왜 이렇게

선택	이유
얇은 층으로 묶음	순수 계산(104~113)은 그대로 두고, 실제 소리·시계만 여기서 얹습니다. 계산 로직을 다시 시험할 필요가 없습니다
`context` 주입 가능	Node에서 가짜 컨텍스트로 배선(예약·클릭·resume)까지 시험합니다. 실제 소리는 브라우저에서 확인
`clock`을 `context.currentTime`으로	오디오 시계는 오실레이터 예약과 같은 시계라 소리가 어긋나지 않습니다
`onStep`을 트랜스포트에 넘김	페이지가 악기 소리를 얹을 자리입니다. 메트로놈을 먼저 울린 뒤 페이지 `onStep`을 부릅니다
클릭 소리를 오실레이터로	샘플 파일 없이 짧은 사각파로 "딱" 소리를 냅니다. 강박은 1600Hz·큰 소리로 구분
`start`에서 `resume`	브라우저는 사용자 조작 뒤에야 소리가 나므로, 멈춰 있으면 깨웁니다. 실패해도 무시하고 진행
카운트인을 `start` 옵션으로	트랜스포트를 음수 위치에서 시작하면 111의 음수 격자가 그대로 카운트인 클릭을 만듭니다
`positionTick` 전달	재생선 그리기는 페이지가 `requestAnimationFrame`으로 이 값을 읽습니다

4. 시험 (Node에서 직접 실행, 41개 통과 · 가짜 AudioContext·가짜 타이머)

확인	결과
`createClickPlayer`: 오실레이터 생성·주파수(1600/1000)·시작/정지 시각·gain 연결	✓
`context` 없이 브라우저도 아니면 `Error`	✓
엔진 구성: 트랜스포트·메트로놈·destination·기본 꺼짐	✓
`start`: suspended → `resume` 1회, 스텝 예약(tick 0·startDelay), 다음 스텝 tick 240	✓
메트로놈 켜면 박마다 클릭, 첫 클릭 강박, 클릭 간격 0.5초(120bpm 4/4)	✓
메트로놈 끄면 클릭 없음, 그래도 스텝은 예약	✓
카운트인: 첫 스텝 −3840(한 마디 전), 강박 클릭	✓
`setTempo`·`setLoop`·`setTimeSignature` 전달, self 반환	✓
`pause`·`resume`·`seek`·`stop` 동작·self 반환	✓